

REPORT

SuperMotors Database System – Final Project Report

Course: CPIS-240 Database Management Systems

Instructor: Dr. Muhammad Binsawad

Term: Fall 2025

Name	ID
Osama Aljabri	2339691
Faisal Alnahdi	2238665
Naif Alnajjar	2340789
Fawaz Yasser Abdelhadi	2343406

Table of Contents

1. Executive Summary	2
2. Introduction	2
3. System Objectives	2
4. Methodological Assumptions	3
5. Conceptual Model (ER Diagram)	3
6. Relational Model (Schema)	4
7. Entities and Attributes (Data Dictionary)	5
8. Relationship Mapping	9
9. SQL Implementation (DDL)	10
10. Sample Data (INSERT Statements)	17
11. Example Queries (Business Use Cases)	20
12. Reports & Stored Calculations	23
13. Security, Backup & Maintenance	24
14. Conclusion	25
15. Appendix A — Additional SQL Examples	26
16. Appendix B — ER DIAGRAM	28

1. Executive Summary

This project delivers a complete, normalized relational database design for **SuperMotors**, a car dealership that sells new and used cars and provides maintenance and repair services. The system stores customers, vehicles, sales, after-sales service tickets, parts usage, employees (salespersons and mechanics), invoices, and related transactions. The design supports multi-salesperson commissions, parts tracking, service assignments, invoice aggregation, and simple payroll estimation.

This report includes: a conceptual ER diagram (placeholder for insertion), the relational schema, DDL (SQL) implementation scripts compatible with MySQL/PostgreSQL, sample data, common queries for reporting and business needs, and recommendations for security and maintenance.

2. Introduction

SuperMotors needs an integrated database system that combines sales and after-sales service data to improve operational tracking, reporting, and payroll calculations. The database must record vehicle inventory, customer details, the details of sales invoices that can include multiple vehicles, service tickets for maintenance tasks, parts used per service, and employee involvement for both sales and repair tasks.

This report documents the database structure, explains entity purposes, defines relationships and constraints, and provides SQL code and sample queries that demonstrate how to use the system for real business tasks such as calculating commissions, producing service histories, and producing consolidated invoices.

3. System Objectives

1. Store and manage customer, vehicle, sales, invoice, service, parts, salesperson, and mechanic data.
2. Track each service operation and which mechanics and parts were used.
3. Track multi-salesperson participation and compute commissions automatically.
4. Support aggregated sales invoices and aggregated service invoices that may include several items/tickets.

5. Provide queries and reports for payroll estimation, sales performance, service history, inventory status, and parts consumption.
6. Ensure referential integrity, enforce business rules, and keep historical records for reporting.

4. Methodological Assumptions

- Each vehicle is uniquely identified by its Vehicle Identification Number (**VIN**) and VIN is used as the natural key for the Car entity.
- A vehicle's status is either inventory (unsold) or sold. When sold, the car record points to the buyer customer.
- Up to three salespersons may participate in a sale of a single car. Each participating salesperson receives a commission (default 2% per company rule). The commission rate is stored or may be defined as a business constant.
- A sale invoice (SaleTransaction) may include multiple cars — each car appears as a SaleLine record linked to the invoice.
- A service invoice may cover multiple service tickets — ServiceInvoice aggregates ServiceTicket records.
- Spare parts used during services are stored in Part and linked to service tickets by a junction table ServicePart to record quantity and cost.
- Employee hire date is stored for computing years of service for salary estimation.
- SQL syntax in the examples is standard and compatible with MySQL and PostgreSQL with minor differences (AUTO_INCREMENT vs SERIAL). Adjust types if necessary for your chosen RDBMS.

5. Conceptual Model (ER Diagram)

Placeholders for diagrams:

- [INSERT ER DIAGRAM HERE]
- [INSERT RELATIONSHIP DIAGRAM / UML DIAGRAM HERE]

Short explanation for the ER diagram:

- Entities: Customer, Car, Salesperson, Mechanic, SaleTransaction, SaleLine, SaleCarSalesperson (junction), ServiceTicket, ServiceAssignment (junction), Part, ServicePart (junction), ServiceInvoice, ServiceInvoiceTicket (junction).
- Key relationships: One Customer may have many SaleTransaction and many ServiceInvoice. One SaleTransaction may have many SaleLine. Each SaleLine may be linked to 0..3 Salesperson via SaleCarSalesperson. A ServiceTicket may be assigned to multiple Mechanic records and involve multiple Part records.

6. Relational Model (Schema)

Below is the high-level relational schema (attributes only). Primary keys are in **bold** and foreign keys are indicated with arrows.

- **Customer**(customer_id, name, phone, email, address, date_of_birth)
- **Car**(vin, make, model, year, color, mileage, purchase_price, sale_price, status, purchased_by)
- **Salesperson**(salesperson_id, name, phone, email, hire_date, basic_salary_base)
- **SaleTransaction**(sale_id, sale_date, invoice_number, customer_id, total_amount)
- **SaleLine**(sale_line_id, sale_id, car_vin, sale_price)
- **SaleCarSalesperson**(sale_line_id, salesperson_id, commission_pct)
- **Mechanic**(mechanic_id, name, phone, specialization, hire_date)
- **ServiceTicket**(ticket_id, ticket_date, car_vin, customer_id, description, status, total_cost)
- **ServiceAssignment**(ticket_id, mechanic_id, hours_worked, notes)
- **Part**(part_id, part_name, part_number, price, stock_qty)
- **ServicePart**(ticket_id, part_id, qty, part_cost)
- **ServiceInvoice**(service_invoice_id, invoice_date, customer_id, total_amount)
- **ServiceInvoiceTicket**(service_invoice_id, ticket_id)

7. Entities and Attributes (Data Dictionary)

Customer

- **customer_id** (PK) — INT AUTO_INCREMENT
- **name** — VARCHAR(255)
- **phone** — VARCHAR(50)
- **email** — VARCHAR(255)
- **address** — TEXT
- **date_of_birth** — DATE

Purpose: Keep personal and contact details. Used for both sales and services.

Car

- **vin** (PK) — VARCHAR(50)
- **make** — VARCHAR(100)
- **model** — VARCHAR(100)
- **year** — INT
- **color** — VARCHAR(50)
- **mileage** — INT
- **purchase_price** — DECIMAL(12,2)
- **sale_price** — DECIMAL(12,2) (nullable — filled when sold)
- **status** — ENUM('inventory','sold') or VARCHAR(20)
- **purchased_by** — INT (FK → Customer.customer_id) — nullable

Purpose: Store vehicle inventory and sale details.

Salesperson

- **salesperson_id** (PK) — INT AUTO_INCREMENT
- **name** — VARCHAR(255)

- **phone** — VARCHAR(50)
- **email** — VARCHAR(255)
- **hire_date** — DATE
- **basic_salary_base** — DECIMAL(12,2) (optional; default company base if null)

Salary formula (business rule):

- $\text{basic_salary} = 10000 + 2000 * \text{years_employed}$
- $\text{commission} = \text{SUM}(\text{sale_price} * \text{commission_pct})$
- $\text{total_salary} = \text{basic_salary} + \text{commission}$

Adjust figures to local currency scales as required.

SaleTransaction

- **sale_id** (PK) — INT AUTO_INCREMENT
- **sale_date** — DATE
- **invoice_number** — VARCHAR(50) UNIQUE
- **customer_id** — INT (FK → Customer.customer_id)
- **total_amount** — DECIMAL(12,2)

Purpose: Represents a sales invoice which may include multiple vehicles (SaleLine rows).

SaleLine

- **sale_line_id** (PK) — INT AUTO_INCREMENT
- **sale_id** — INT (FK → SaleTransaction.sale_id)
- **car_vin** — VARCHAR(50) (FK → Car.vin)
- **sale_price** — DECIMAL(12,2)

Purpose: Represents each vehicle sold within a sale transaction.

SaleCarSalesperson

- **sale_line_id** — INT (FK → SaleLine.sale_line_id)
- **salesperson_id** — INT (FK → Salesperson.salesperson_id)
- **commission_pct** — DECIMAL(5,4) DEFAULT 0.02
- **PRIMARY KEY**(sale_line_id, salesperson_id)

Purpose: Junction table to record which salespersons participated in selling a particular vehicle and their commission percentage.

Mechanic

- **mechanic_id** (PK) — INT AUTO_INCREMENT
- **name** — VARCHAR(255)
- **phone** — VARCHAR(50)
- **specialization** — VARCHAR(255)
- **hire_date** — DATE

Purpose: Track mechanics who perform service tasks.

ServiceTicket

- **ticket_id** (PK) — INT AUTO_INCREMENT
- **ticket_date** — DATE
- **car_vin** — VARCHAR(50) (FK → Car.vin)
- **customer_id** — INT (FK → Customer.customer_id)
- **description** — TEXT
- **status** — ENUM('open','in_progress','completed','closed') or VARCHAR(50)
- **total_cost** — DECIMAL(12,2)

Purpose: Each maintenance or repair request creates a service ticket for a vehicle.

ServiceAssignment

- **ticket_id** — INT (FK → ServiceTicket.ticket_id)

- **mechanic_id** — INT (FK → Mechanic.mechanic_id)
- **hours_worked** — DECIMAL(5,2)
- **notes** — TEXT
- **PRIMARY KEY**(ticket_id, mechanic_id)

Purpose: Junction table to assign mechanics and record time worked per ticket.

Part

- **part_id** (PK) — INT AUTO_INCREMENT
- **part_name** — VARCHAR(255)
- **part_number** — VARCHAR(100) UNIQUE
- **price** — DECIMAL(12,2)
- **stock_qty** — INT

Purpose: Inventory of spare parts used in services.

ServicePart

- **ticket_id** — INT (FK → ServiceTicket.ticket_id)
- **part_id** — INT (FK → Part.part_id)
- **qty** — INT
- **part_cost** — DECIMAL(12,2)
- **PRIMARY KEY**(ticket_id, part_id)

Purpose: Record parts consumed per service ticket and their cost.

ServiceInvoice

- **service_invoice_id** (PK) — INT AUTO_INCREMENT
- **invoice_date** — DATE
- **customer_id** — INT (FK → Customer.customer_id)
- **total_amount** — DECIMAL(12,2)

Purpose: A service invoice that can aggregate multiple service tickets for billing the customer.

ServiceInvoiceTicket

- **service_invoice_id** — INT (FK → ServiceInvoice.service_invoice_id)
- **ticket_id** — INT (FK → ServiceTicket.ticket_id)
- **PRIMARY KEY**(service_invoice_id, ticket_id)

Purpose: Junction table linking service invoices to the service tickets included on the invoice.

8. Relationship Mapping (Narrative)

1. **Customer — SaleTransaction:** One-to-many. A customer may have many sale invoices.
2. **SaleTransaction — SaleLine:** One-to-many. Each sale invoice may contain multiple sale lines (vehicles).
3. **SaleLine — SaleCarSalesperson — Salesperson:** Many-to-many via junction table. Allows multiple salespersons per sold vehicle. Commission percentage recorded in junction.
4. **Car — SaleLine:** One-to-one per sold car — each SaleLine references one car via VIN. A car may appear once in sale history (historical) but could be in inventory prior to sale.
5. **Customer — ServiceInvoice:** One-to-many. A customer may have multiple service invoices.
6. **ServiceInvoice — ServiceTicket:** Many-to-many via ServiceInvoiceTicket (one service invoice may include several tickets).
7. **ServiceTicket — ServiceAssignment — Mechanic:** Many-to-many. A service ticket can have multiple mechanics assigned.
8. **ServiceTicket — ServicePart — Part:** Many-to-many. A ticket may use many parts; parts can be used in many tickets.

Referential integrity rules:

- Deleting a customer should be restricted if they have associated sales or services (use ON DELETE RESTRICT or archive strategy).
- Deleting a car should be restricted if it appears in sale or service history.
- When a sale is recorded, the Car.status should be updated to sold and purchased_by set to the buyer.

9. SQL Implementation (DDL)

Below is a MySQL-compatible DDL script. For PostgreSQL adjust AUTO_INCREMENT to SERIAL and change ENUM usage or use CHECK constraints.

-- Drop tables if exist (safe re-run)

DROP TABLE IF EXISTS ServiceInvoiceTicket;

DROP TABLE IF EXISTS ServiceInvoice;

DROP TABLE IF EXISTS ServicePart;

DROP TABLE IF EXISTS Part;

DROP TABLE IF EXISTS ServiceAssignment;

DROP TABLE IF EXISTS ServiceTicket;

DROP TABLE IF EXISTS SaleCarSalesperson;

DROP TABLE IF EXISTS SaleLine;

DROP TABLE IF EXISTS SaleTransaction;

DROP TABLE IF EXISTS Salesperson;

DROP TABLE IF EXISTS Mechanic;

DROP TABLE IF EXISTS Car;

DROP TABLE IF EXISTS Customer;

-- Customer

```
CREATE TABLE Customer (  
    customer_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    phone VARCHAR(50),  
    email VARCHAR(255),  
    address TEXT,  
    date_of_birth DATE  
);
```

-- Car

```
CREATE TABLE Car (  
    vin VARCHAR(50) PRIMARY KEY,  
    make VARCHAR(100),  
    model VARCHAR(100),  
    year INT,  
    color VARCHAR(50),  
    mileage INT,  
    purchase_price DECIMAL(12,2),  
    sale_price DECIMAL(12,2),  
    status VARCHAR(20) DEFAULT 'inventory',  
    purchased_by INT,  
    CONSTRAINT fk_car_customer FOREIGN KEY (purchased_by) REFERENCES  
    Customer(customer_id) ON DELETE SET NULL
```

);

-- Salesperson

```
CREATE TABLE Salesperson (  
    salesperson_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    phone VARCHAR(50),  
    email VARCHAR(255),  
    hire_date DATE,  
    basic_salary_base DECIMAL(12,2) DEFAULT NULL  
);
```

-- SaleTransaction (Sales Invoice)

```
CREATE TABLE SaleTransaction (  
    sale_id INT AUTO_INCREMENT PRIMARY KEY,  
    sale_date DATE NOT NULL,  
    invoice_number VARCHAR(50) NOT NULL UNIQUE,  
    customer_id INT,  
    total_amount DECIMAL(12,2),  
    CONSTRAINT fk_sale_customer FOREIGN KEY (customer_id) REFERENCES  
    Customer(customer_id) ON DELETE SET NULL  
);
```

-- SaleLine (vehicles within a sale)

```

CREATE TABLE SaleLine (
    sale_line_id INT AUTO_INCREMENT PRIMARY KEY,
    sale_id INT NOT NULL,
    car_vin VARCHAR(50) NOT NULL,
    sale_price DECIMAL(12,2) NOT NULL,
    CONSTRAINT fk_saleline_sale FOREIGN KEY (sale_id) REFERENCES
SaleTransaction(sale_id) ON DELETE CASCADE,
    CONSTRAINT fk_saleline_car FOREIGN KEY (car_vin) REFERENCES Car(vin) ON
DELETE RESTRICT
);

```

-- SaleCarSalesperson (junction)

```

CREATE TABLE SaleCarSalesperson (
    sale_line_id INT NOT NULL,
    salesperson_id INT NOT NULL,
    commission_pct DECIMAL(5,4) DEFAULT 0.02,
    PRIMARY KEY (sale_line_id, salesperson_id),
    CONSTRAINT fk_scs_saleline FOREIGN KEY (sale_line_id) REFERENCES
SaleLine(sale_line_id) ON DELETE CASCADE,
    CONSTRAINT fk_scs_salesperson FOREIGN KEY (salesperson_id) REFERENCES
Salesperson(salesperson_id) ON DELETE CASCADE
);

```

-- Mechanic

```

CREATE TABLE Mechanic (

```

```
mechanic_id INT AUTO_INCREMENT PRIMARY KEY,  
name VARCHAR(255) NOT NULL,  
phone VARCHAR(50),  
specialization VARCHAR(255),  
hire_date DATE  
);
```

-- ServiceTicket

```
CREATE TABLE ServiceTicket (  
    ticket_id INT AUTO_INCREMENT PRIMARY KEY,  
    ticket_date DATE NOT NULL,  
    car_vin VARCHAR(50) NOT NULL,  
    customer_id INT,  
    description TEXT,  
    status VARCHAR(50) DEFAULT 'open',  
    total_cost DECIMAL(12,2) DEFAULT 0,  
    CONSTRAINT fk_ticket_car FOREIGN KEY (car_vin) REFERENCES Car(vin) ON  
DELETE RESTRICT,  
    CONSTRAINT fk_ticket_customer FOREIGN KEY (customer_id) REFERENCES  
Customer(customer_id) ON DELETE SET NULL  
);
```

-- ServiceAssignment (junction)

```
CREATE TABLE ServiceAssignment (
```

```

ticket_id INT NOT NULL,
mechanic_id INT NOT NULL,
hours_worked DECIMAL(5,2) DEFAULT 0,
notes TEXT,
PRIMARY KEY (ticket_id, mechanic_id),
CONSTRAINT fk_sa_ticket FOREIGN KEY (ticket_id) REFERENCES
ServiceTicket(ticket_id) ON DELETE CASCADE,
CONSTRAINT fk_sa_mechanic FOREIGN KEY (mechanic_id) REFERENCES
Mechanic(mechanic_id) ON DELETE CASCADE
);

```

-- Part

```

CREATE TABLE Part (
    part_id INT AUTO_INCREMENT PRIMARY KEY,
    part_name VARCHAR(255) NOT NULL,
    part_number VARCHAR(100) UNIQUE,
    price DECIMAL(12,2) NOT NULL,
    stock_qty INT DEFAULT 0
);

```

-- ServicePart (junction)

```

CREATE TABLE ServicePart (
    ticket_id INT NOT NULL,
    part_id INT NOT NULL,

```



```

    qty INT DEFAULT 1,
    part_cost DECIMAL(12,2) DEFAULT 0,
    PRIMARY KEY (ticket_id, part_id),
    CONSTRAINT fk_sp_ticket FOREIGN KEY (ticket_id) REFERENCES
ServiceTicket(ticket_id) ON DELETE CASCADE,
    CONSTRAINT fk_sp_part FOREIGN KEY (part_id) REFERENCES Part(part_id) ON
DELETE RESTRICT
);

```

-- ServiceInvoice

```

CREATE TABLE ServiceInvoice (
    service_invoice_id INT AUTO_INCREMENT PRIMARY KEY,
    invoice_date DATE NOT NULL,
    customer_id INT,
    total_amount DECIMAL(12,2),
    CONSTRAINT fk_si_customer FOREIGN KEY (customer_id) REFERENCES
Customer(customer_id) ON DELETE SET NULL
);

```

-- ServiceInvoiceTicket (junction)

```

CREATE TABLE ServiceInvoiceTicket (
    service_invoice_id INT NOT NULL,
    ticket_id INT NOT NULL,
    PRIMARY KEY (service_invoice_id, ticket_id),

```

```
CONSTRAINT fk_sit_si FOREIGN KEY (service_invoice_id) REFERENCES  
ServiceInvoice(service_invoice_id) ON DELETE CASCADE,
```

```
CONSTRAINT fk_sit_ticket FOREIGN KEY (ticket_id) REFERENCES  
ServiceTicket(ticket_id) ON DELETE RESTRICT
```

```
);
```

Implementation notes:

- For PostgreSQL, replace AUTO_INCREMENT with SERIAL or GENERATED ALWAYS AS IDENTITY and convert VARCHAR sizes appropriately.
- Consider adding indexes on frequently queried columns such as Car.status, SaleTransaction.sale_date, ServiceTicket.ticket_date, and Part.part_number.

10. Sample Data (INSERT Statements)

Below are sample INSERT statements aligned with the DDL above. Replace values as needed.

-- Customers

```
INSERT INTO Customer (name, phone, email, address, date_of_birth) VALUES
```

```
('Ahmed Ali', '+966500000001', 'ahmed@example.com', 'Riyadh, Saudi Arabia', '1985-04-01'),
```

```
('Fatima Saleh', '+966500000002', 'fatima@example.com', 'Jeddah, Saudi Arabia', '1990-09-10');
```

-- Cars

```
INSERT INTO Car (vin, make, model, year, color, mileage, purchase_price, status)  
VALUES
```

```
('VIN0001', 'Toyota', 'Corolla', 2018, 'White', 45000, 30000, 'inventory'),
```

```
('VIN0002', 'Honda', 'Civic', 2017, 'Black', 60000, 28000, 'inventory');
```

-- Salespersons

```
INSERT INTO Salesperson (name, phone, email, hire_date, basic_salary_base)
VALUES
```

```
('Saleem', '+966500100001', 'saleem@super.com', '2018-03-15', NULL),
```

```
('Nora', '+966500100002', 'nora@super.com', '2020-07-01', NULL),
```

```
('Omar', '+966500100003', 'omar@super.com', '2022-01-05', NULL);
```

-- Sale Transaction and Sale Lines (example: invoice INV-1001 selling two cars)

```
INSERT INTO SaleTransaction (sale_date, invoice_number, customer_id, total_amount)
VALUES ('2025-10-30', 'INV-1001', 1, 60000);
```

-- Assume auto-increment returned id = 1 for sale_id. Use LAST_INSERT_ID() in scripts or fetch id in application.

-- SaleLines for that sale

```
INSERT INTO SaleLine (sale_id, car_vin, sale_price) VALUES
```

```
(1, 'VIN0001', 31000),
```

```
(1, 'VIN0002', 29000);
```

-- Link salespersons to sale lines (multiple salespersons can be linked; commission defaults to 2%)

```
INSERT INTO SaleCarSalesperson (sale_line_id, salesperson_id, commission_pct)
VALUES
```

```
(1, 1, 0.02),
```

```
(1, 2, 0.02),
```

```
(2, 3, 0.02);
```

-- Mechanics

```
INSERT INTO Mechanic (name, phone, specialization, hire_date) VALUES  
(('Khaled', '+966500200001', 'Engine Specialist', '2017-06-20'),  
(('Mona', '+966500200002', 'Transmission Specialist', '2019-11-01'));
```

-- Parts

```
INSERT INTO Part (part_name, part_number, price, stock_qty) VALUES  
(('Oil Filter', 'OF-001', 15.00, 150),  
(('Brake Pad Set', 'BP-100', 75.00, 40));
```

-- Service Tickets

```
INSERT INTO ServiceTicket (ticket_date, car_vin, customer_id, description, status,  
total_cost) VALUES  
(('2025-11-05', 'VIN0001', 1, 'Oil change and general inspection', 'completed', 200.00));
```

-- ServiceAssignment

```
INSERT INTO ServiceAssignment (ticket_id, mechanic_id, hours_worked, notes)  
VALUES  
(1, 1, 2.5, 'Oil change and filter replacement');
```

-- ServicePart (parts used on ticket 1)

```
INSERT INTO ServicePart (ticket_id, part_id, qty, part_cost) VALUES  
(1, 1, 1, 15.00);
```

-- Create a service invoice that includes ticket 1

```
INSERT INTO ServiceInvoice (invoice_date, customer_id, total_amount) VALUES  
(2025-11-06, 1, 200.00);
```

-- Link service invoice to ticket

```
INSERT INTO ServiceInvoiceTicket (service_invoice_id, ticket_id) VALUES  
(1, 1);
```

-- Update Car status after sale (application logic or trigger can do this)

```
UPDATE Car SET status = 'sold', purchased_by = 1, sale_price = 31000 WHERE vin =  
'VIN0001';
```

```
UPDATE Car SET status = 'sold', purchased_by = 1, sale_price = 29000 WHERE vin =  
'VIN0002';
```

11. Example Queries (Business Use Cases)

11.1 Cars sold in a specific invoice (with salespersons)

```
SELECT st.invoice_number,  
       sl.sale_line_id,  
       sl.car_vin,  
       sl.sale_price,  
       GROUP_CONCAT(scs.salesperson_id) AS salesperson_ids  
FROM SaleTransaction st  
JOIN SaleLine sl ON st.sale_id = sl.sale_id
```

```
LEFT JOIN SaleCarSalesperson scs ON sl.sale_line_id = scs.sale_line_id
```

```
WHERE st.invoice_number = 'INV-1001'
```

```
GROUP BY sl.sale_line_id, st.invoice_number, sl.car_vin, sl.sale_price;
```

Explanation: Retrieves all cars listed on invoice INV-1001 and the IDs of salespersons associated with each sale line.

11.2 Salesperson commission for October 2025

```
SELECT s.salesperson_id,
```

```
       s.name,
```

```
       SUM(sl.sale_price * scs.commission_pct) AS total_commission
```

```
FROM Salesperson s
```

```
JOIN SaleCarSalesperson scs ON s.salesperson_id = scs.salesperson_id
```

```
JOIN SaleLine sl ON scs.sale_line_id = sl.sale_line_id
```

```
JOIN SaleTransaction st ON sl.sale_id = st.sale_id
```

```
WHERE st.sale_date BETWEEN '2025-10-01' AND '2025-10-31'
```

```
GROUP BY s.salesperson_id, s.name;
```

Explanation: Calculates commissions for each salesperson using the commission_pct saved in the junction table.

11.3 Total salary (base + commission) for the year 2025

```
SELECT s.salesperson_id,
```

```
       s.name,
```

```
       (10000 + 2000 * TIMESTAMPDIFF(YEAR, s.hire_date, CURDATE())) AS  
basic_salary,
```

```
       COALESCE(comm.total_commission, 0) AS commission,
```

```
       (10000 + 2000 * TIMESTAMPDIFF(YEAR, s.hire_date, CURDATE())) +  
COALESCE(comm.total_commission, 0)) AS total_salary
```

```

FROM Salesperson s
LEFT JOIN (
    SELECT sps.salesperson_id, SUM(sl.sale_price * sps.commission_pct) AS
    total_commission
    FROM SaleCarSalesperson sps
    JOIN SaleLine sl ON sps.sale_line_id = sl.sale_line_id
    JOIN SaleTransaction st ON sl.sale_id = st.sale_id
    WHERE st.sale_date BETWEEN '2025-01-01' AND '2025-12-31'
    GROUP BY sps.salesperson_id
) comm ON s.salesperson_id = comm.salesperson_id;

```

Explanation: Computes a dynamic basic salary based on hire date plus commission for the calendar year.

11.4 Maintenance history for a car (with mechanics and parts)

```

SELECT t.ticket_id,
       t.ticket_date,
       t.description,
       t.total_cost,
       GROUP_CONCAT(DISTINCT m.name) AS mechanics,
       GROUP_CONCAT(CONCAT(p.part_name, ' (', sp.qty, ')')) AS parts_used
FROM ServiceTicket t
LEFT JOIN ServiceAssignment sa ON t.ticket_id = sa.ticket_id
LEFT JOIN Mechanic m ON sa.mechanic_id = m.mechanic_id
LEFT JOIN ServicePart sp ON t.ticket_id = sp.ticket_id
LEFT JOIN Part p ON sp.part_id = p.part_id

```

WHERE t.car_vin = 'VIN0001'

GROUP BY t.ticket_id, t.ticket_date, t.description, t.total_cost;

Explanation: Returns service tickets for a particular VIN, listing involved mechanics and parts used.

11.5 Top salesperson by total sales

SELECT s.salesperson_id,

s.name,

SUM(sl.sale_price) AS total_sales

FROM Salesperson s

JOIN SaleCarSalesperson scs ON s.salesperson_id = scs.salesperson_id

JOIN SaleLine sl ON scs.sale_line_id = sl.sale_line_id

GROUP BY s.salesperson_id, s.name

ORDER BY total_sales DESC

LIMIT 1;

Explanation: Identifies the salesperson with the greatest total value of sales (based on sale lines).

12. Reports & Stored Calculations

Suggested reports and views:

1. **Monthly Sales Report:** total invoiced sales, number of vehicles sold, top models, salesperson rankings.
2. **Service Revenue Report:** revenue from services, parts usage, mechanic utilization.
3. **Inventory Report:** list of cars with status='inventory', days in inventory.
4. **Parts Stock Report:** parts low in stock (stock_qty < reorder_level).

5. **Employee Payroll Summary:** basic salary calculations and commissions.

Sample view for monthly sales summary:

```
CREATE VIEW vw_monthly_sales AS
SELECT DATE_FORMAT(st.sale_date, '%Y-%m') AS sale_month,
       COUNT(DISTINCT st.sale_id) AS invoices_count,
       COUNT(sl.sale_line_id) AS vehicles_sold,
       SUM(sl.sale_price) AS total_revenue
FROM SaleTransaction st
JOIN SaleLine sl ON st.sale_id = sl.sale_id
GROUP BY DATE_FORMAT(st.sale_date, '%Y-%m');
```

Automation:

- Use stored procedures to compute and store commissions at month-end.
- Use triggers to reduce Part.stock_qty when ServicePart records are inserted.

13. Security, Backup & Maintenance

Security recommendations:

- Use role-based access control: create roles (admin, sales, mechanic, accountant) and grant minimal privileges.
- Encrypt sensitive data at rest if required by policy (customer PII).
- Use TLS for client-server connections.
- Regularly audit access and log modifications to financial tables.

Backup & maintenance:

- Daily incremental backups, weekly full backups.
- Test restore procedures monthly.

- Monitor indices and vacuum/rebuild (for PostgreSQL) to maintain performance.
- Archive old sales and service records into read-only historical tables if the production size grows substantially.

14. Conclusion

The design provided supports SuperMotors' core operational needs: vehicle inventory and sale tracking, multi-salesperson commission handling, service ticket management with parts and mechanics tracking, and invoice aggregation. The schema is normalized, enforces referential integrity, and includes sample SQL, data, and queries needed for reporting and payroll calculations. Application-level transactions and a small set of triggers/stored procedures are recommended to keep data consistent (for example: updating car status after sale, decrementing part stock after use).

15. Appendix A — Additional SQL Examples

Trigger example: update Car status after SaleLine insertion (MySQL)

```
DELIMITER $$
```

```
CREATE TRIGGER trg_after_saleline_insert
```

```
AFTER INSERT ON SaleLine
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    UPDATE Car
```

```
    SET status = 'sold', purchased_by = (SELECT customer_id FROM SaleTransaction  
    WHERE sale_id = NEW.sale_id),
```

```
        sale_price = NEW.sale_price
```

```
    WHERE vin = NEW.car_vin;
```

```
END$$
```

```
DELIMITER ;
```

Trigger example: decrement Part stock after ServicePart insert

```
DELIMITER $$
```

```
CREATE TRIGGER trg_after_servicepart_insert
```

```
AFTER INSERT ON ServicePart
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    UPDATE Part
```

```
    SET stock_qty = GREATEST(stock_qty - NEW.qty, 0)
```

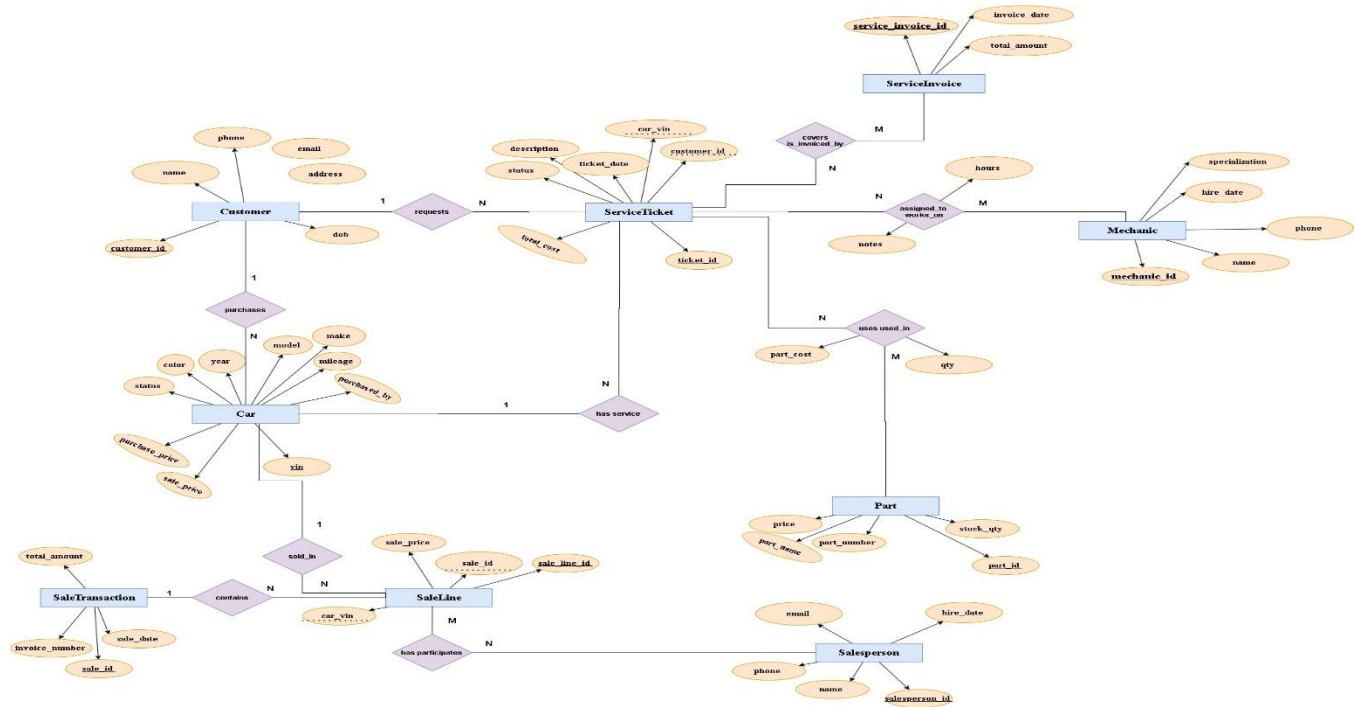
```
    WHERE part_id = NEW.part_id;
```

```
END$$
```

DELIMITER ;

16. Appendix B — ER DIAGRAM

ER DIAGRAM



RELATIONSHIP SCHEMA MAPPING

